Project - High Level Design

on

Real Estate Quiz/Tutor Bot

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	HARSHIT MITTAL	EN22CS301401
2	HARSHAL DESHMUKH	EN22CS301396
3	HARSHIT BHARDWAJ	EN22CS301399
4	HARSHIT CHOUDHARY	EN22CS301400
5	HARSHIT SHARMA	EN22CS301403
6	SALONEE CHOUHAN	EN23CS3L1017

Group Name: Group 11D4

Project Number: GAI-47

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces
   4.1. User Interface
   4.2. External Interface
   4.3. Database Interface
   4.4. Internal Interface

5. State and Session Management
   5.1. Application State Management
   5.2. Session Handling

6. Caching
   6.1. Overview
   6.2. Embedding's Caching
   6.3. Streamlit's Caching Mechanism
   6.4. AI Response Caching
   6.5. Design Consideration

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1) INTRODUCTION

The Real Estate Tutor Bot is an AI-powered learning system designed to help users understand real estate concepts through interactive, property-based multiple-choice questions (MCQs). Users upload property listings in JSON format, which are embedded and stored in ChromaDB for context-aware retrieval.

The system uses a Retrieval-Augmented Generation (RAG) approach powered by **Gemini 2.5 Flash** and the 001 embedding model to generate questions, evaluate answers, and provide explanations, industry insights, and follow-up study recommendations.

Developed using Streamlit and core Python, the application functions without a separate backend API. It adapts content based on the user's selected experience level and response preference, making it an intelligent and personalized real estate learning platform.

1.1 Scope of the document

This document provides the High-Level Design (HLD) of the Real Estate Tutor Bot system. It describes the overall architecture, system components, data flow, design considerations, and key technical decisions involved in the development of the platform.

The document outlines how property listings uploaded in JSON format are processed, embedded, stored in ChromaDB, and utilized through a Retrieval-Augmented Generation (RAG) approach to generate AI-based multiple-choice questions. It also explains how user-selected parameters such as experience level and response type influence question generation and feedback mechanisms.

This HLD focuses on the structural and architectural aspects of the system rather than low-level implementation details or source code. It serves as a reference for developers, reviewers, mentors, and stakeholders to understand the overall system design and functionality.

1.2 Intended Audience

This document is intended for project evaluators, university mentors, industry mentors, and development team members who require an understanding of the system's overall architecture and design.

It is particularly useful for:

- Faculty members reviewing the technical design of the project
- Industry mentors evaluating architectural decisions
- Developers who may extend or maintain the system
- Stakeholders interested in understanding the system workflow and AI integration

1.3 System Overview

The Real Estate Tutor Bot is an AI-driven educational web application built using Streamlit and core Python. The system enables users to upload property listings in JSON format, which are processed and stored in ChromaDB using vector embeddings.

The architecture follows a Retrieval-Augmented Generation (RAG) approach. When a user selects their experience level (Beginner, Intermediate, Experienced) and preferred response style (Concise or Detailed), the system retrieves relevant property data from the vector database and uses Gemini 2.5 Flash to dynamically generate multiple-choice questions (MCQs).

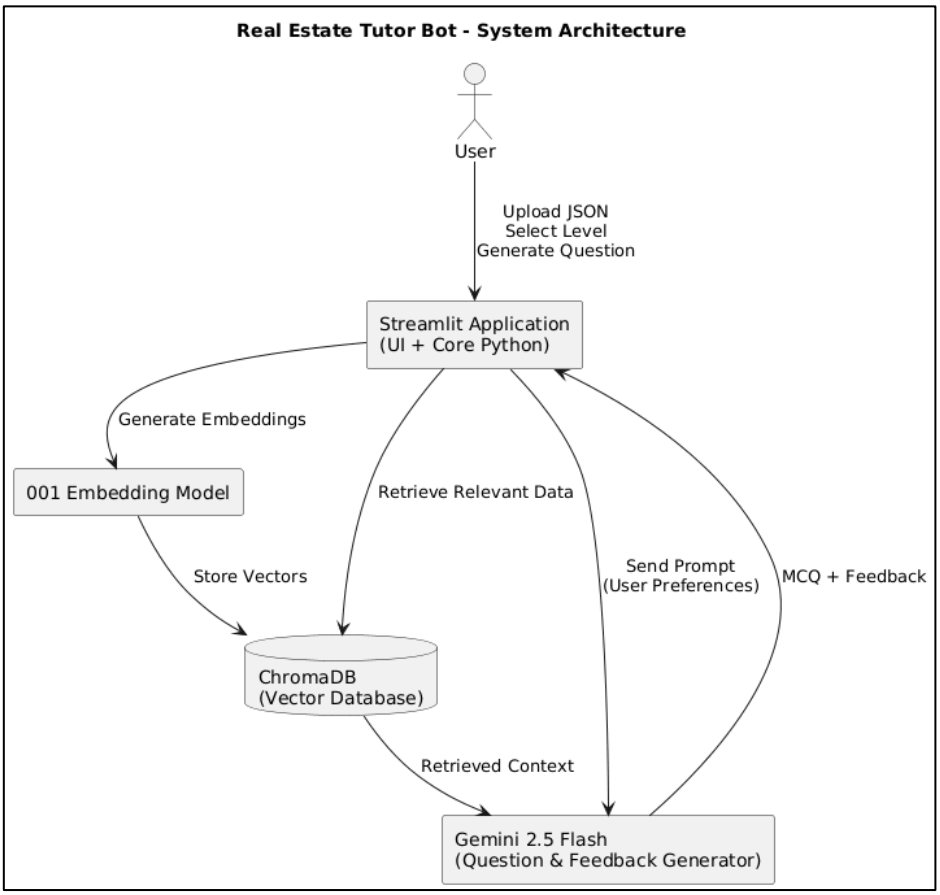
After the user selects an answer, the system evaluates the response and generates structured feedback, including explanations, industry insights, and recommended follow-up study topics.

The application operates without a separate backend API and runs entirely within the Streamlit environment, making it lightweight, modular, and easy to extend for future enhancements.

---

2) SYSTEM DESIGN

The Real Estate Tutor Bot follows a modular, Retrieval-Augmented Generation (RAG)-based architecture implemented using Streamlit and core Python. The system is designed to process user-uploaded property listings, convert them into vector embeddings, and generate AI-driven educational content dynamically.



The diagram illustrates the overall architecture of the Real Estate Tutor Bot, which follows a Retrieval-Augmented Generation (RAG) approach.

The process begins when the **User** interacts with the **Streamlit Application** to upload property listings, select experience level, and generate questions. The application handles both the user interface and backend logic.

Uploaded property data is converted into **vector embeddings** using the **001 Embedding Model** and stored in **ChromaDB**. During question generation, relevant data is retrieved from ChromaDB and sent, along with user preferences, to **Gemini 2.5 Flash**.

Gemini generates the MCQ and structured feedback, which is then displayed back to the **user** through the **Streamlit interface**.

2.1 Application Design

The Real Estate Tutor Bot is designed as a single-tier AI-powered web application built using Streamlit and core Python. The application integrates data processing, vector storage, and AI generation within a unified environment without a separate backend API.

The application design consists of the following logical modules:

1. User Interface Module (Streamlit)

- Upload property listings in JSON format
- Select experience level (Beginner / Intermediate / Experienced)
- Select response style (Concise / Detailed)
- Display generated MCQs and feedback

2. Data Processing Module

- Parses and validates uploaded JSON data
- Extracts relevant property attributes
- Converts property content into embeddings using the 001 embedding model

3. Vector Storage Module (ChromaDB)

- Stores property embeddings
- Performs similarity-based retrieval during question generation

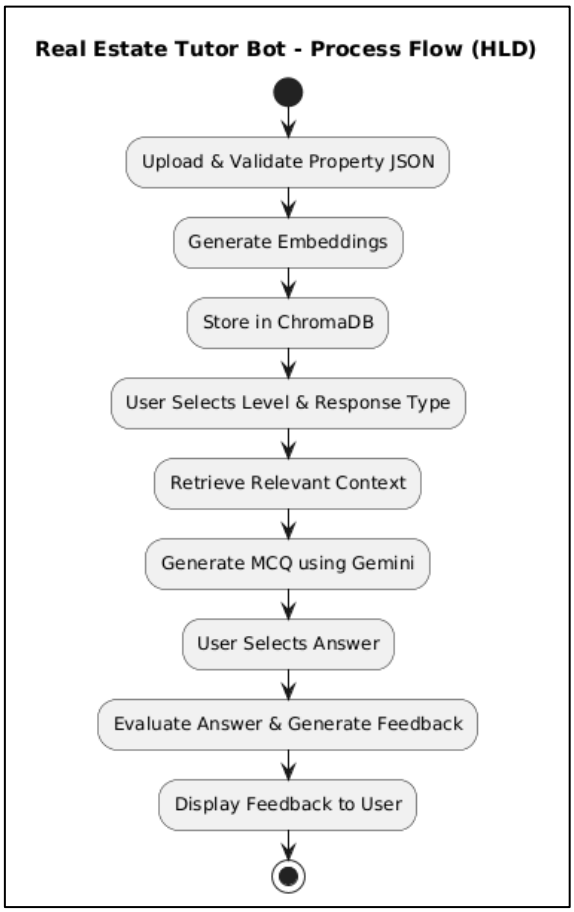
4. AI Interaction Module (Gemini 2.5 Flash)

- Generates MCQ questions based on retrieved context
- Evaluates user-selected answers
- Produces explanations, industry insights, and follow-up study topics

The design ensures modularity, scalability, and easy extensibility for future enhancements such as session management, analytics tracking, or deployment to cloud infrastructure.

2.2 Process Flow

The process flow of the Real Estate Tutor Bot describes how data moves through the system from user input to AI-generated feedback.



Step 1: Property Upload

- The user uploads property listings in JSON format via the Streamlit interface.
- The system validates the JSON structure and extracts relevant fields (description, zoning, legal notes, etc.).

Step 2: Data Processing & Embedding

- Extracted property data is converted into textual format.
- The 001 embedding model generates vector embeddings.
- The embeddings are stored in ChromaDB along with property metadata.

Step 3: User Configuration

- The user selects:
  - Experience Level (Beginner / Intermediate / Experienced)
  - Response Type (Concise / Detailed)

Step 4: Question Generation

- When the user clicks **Generate Question**:
  - Relevant property embeddings are retrieved from ChromaDB using similarity search.
  - Retrieved context + user preferences are sent to Gemini 2.5 Flash.
  - Gemini generates a context-aware MCQ with four options.

Step 5: Answer Evaluation

- The user selects one of the options.
- The selected answer is sent to Gemini.
- Gemini evaluates correctness and generates:
  - Explanation
  - Industry insights
  - Follow-up study topics

Step 6: Feedback Display

- Structured feedback is displayed on the Streamlit interface.

2.3 Information Flow

The Information Flow describes how data is transmitted and transformed across different components of the Real Estate Tutor Bot system.

1. User to Application Layer

The process begins when the user uploads property listings in JSON format and selects configuration options such as experience level and response type. This input data is received and processed by the Streamlit application.

2. Application to Embedding Model

The parsed property data is converted into structured text and sent to the 001 embedding model. The model generates numerical vector representations of the property information.

3. Embedding Model to Vector Database

The generated embeddings, along with associated metadata (property ID, zoning, legal notes, etc.), are stored in ChromaDB for efficient similarity-based retrieval.

4. Retrieval Flow (Vector Database to Application)

When the user requests question generation, the application queries ChromaDB to retrieve the most relevant property vectors based on semantic similarity.

5. Application to AI Model (Gemini 2.5 Flash)

The retrieved contextual data, combined with user preferences (experience level and response style), is sent to Gemini 2.5 Flash. The AI model processes this information to generate:

- A multiple-choice question
- Four answer options
- Evaluation logic

6. Feedback Flow

After the user selects an answer, the selected option is sent back to Gemini for evaluation. The model generates structured feedback including explanation, industry insights, and follow-up study recommendations. This output is returned to the Streamlit application and displayed to the user.

2.4 Components Design

The Real Estate Tutor Bot is designed using modular components to ensure separation of concerns, maintainability, and scalability. Each component performs a specific responsibility within the overall system.

1. User Interface Component (Streamlit)

This component manages user interaction and presentation logic. It allows users to:

- Upload property listings in JSON format
- Select experience level and response type
- Generate MCQs
- View feedback and explanations

It also coordinates communication between backend modules and displays AI-generated outputs.

2. Data Processing Component

This module handles:

- JSON validation and parsing
- Extraction of relevant property attributes
- Formatting structured property data into text suitable for embedding.

It ensures that only clean and structured data is passed to the embedding model.

3. Embedding Component (001 Embedding Model)

This component converts textual property data into high-dimensional vector representations. These embeddings enable semantic similarity search within the vector database.

4. Vector Storage Component (ChromaDB)

ChromaDB stores:

- Property embeddings
- Associated metadata (ID, zoning, legal notes, etc.)

It supports similarity-based retrieval during question generation.

5. AI Generation Component (Gemini 2.5 Flash)

This component is responsible for:

- Generating MCQ questions
- Evaluating user-selected answers
- Providing explanations
- Generating industry insights
- Suggesting follow-up study topics

It receives contextual data retrieved from ChromaDB along with user preferences.

6. Retrieval Mechanism (RAG Layer)

This logical component integrates the embedding model, vector database, and Gemini model to enable Retrieval-Augmented Generation, ensuring context-aware and accurate question generation.

2.5 Key Design Considerations

- **Modularity** – Separation of UI, data processing, embedding, vector storage, and AI generation components for maintainability and flexibility.
- **Scalability** – Architecture designed to support future cloud deployment and component-level scaling.
- **RAG-Based Design** – Use of Retrieval-Augmented Generation to ensure context-aware and accurate AI-generated questions.
- **Performance Efficiency** – Use of vector embeddings and similarity search for fast data retrieval.
- **User Personalization** – Adaptive question difficulty and response depth based on selected experience level and response type.
- **Data Validation** – JSON input validation to maintain data integrity and system stability.
- **Extensibility** – Easy integration of additional features such as session management, analytics, or advanced AI models in future versions.

2.6 API Catalogue

The Real Estate Tutor Bot uses external AI services and internal module calls for its functionality. Since the application is built using Streamlit with core Python, it does not expose custom REST APIs. The primary APIs used are described below.

1. Gemini 2.5 Flash API

Purpose:

- Generate MCQ questions
- Evaluate selected answers
- Provide explanations, industry insights, and follow-up study topics

Input:

- Retrieved property context
- User-selected experience level
- Response type preference
- Selected answer (for evaluation)

Output:

- MCQ with four options
- Structured feedback and learning recommendations

2. Embedding Model API (001 Embedding)

Purpose:

- Convert property text data into vector embeddings

Input:

- Structured property description text

Output:

- High-dimensional vector representation of the property data

3. ChromaDB Interface (Internal API/Library Calls)

Purpose:

- Store embeddings with metadata
- Perform similarity-based retrieval

Operations Used:

- Add embeddings
- Query similar vectors

3. DATA DESIGN

The Data Design of the Real Estate Tutor Bot defines how property data is structured, stored, processed, and retrieved within the system.

3.1 Data Model

The system accepts property listings in **JSON format**. Each property record contains the following attributes:

- id – Unique identifier for the property (e.g., RES-101, COM-202).
- address – Full physical location of the property.
- description – Detailed narrative about the property, including notable conditions or issues.
- price – Listed market price of the property.
- features – Key specifications such as number of rooms, area, build year, etc.
- zoning – Zoning classification (e.g., Residential, Commercial, Industrial).
- legal_notes – Legal disclosures or compliance-related information.

This structure ensures that both financial and regulatory aspects of real estate are captured.

3.2 Data Access Mechanism

The Data Access Mechanism defines how property data and embeddings are stored, retrieved, and utilized within the system.

1. JSON Data Access

- Uploaded property listings are received through the Streamlit interface.
- The system parses the JSON file using Python's built-in data handling libraries.
- Data is temporarily stored in memory during processing.
- Input validation ensures proper structure before further operations.

2. Embedding Storage Access (ChromaDB)

After text conversion, embeddings are generated and stored in ChromaDB along with associated metadata.

The system performs two primary database operations:

- Insert Operation:
  Adds embedding vectors and metadata into the vector database.
- Query Operation (Similarity Search):
  Retrieves the most relevant property embeddings based on semantic similarity when generating MCQs.

3. Retrieval Mechanism (RAG-Based Access)

When a user requests question generation:

1) The application sends a similarity query to ChromaDB.

2) The top-matching property vectors are retrieved.

3) Retrieved data is passed to Gemini for context-aware question generation.

This ensures that AI outputs are based strictly on relevant property data rather than generic responses.

4. AI Model Interaction

The system interacts with Gemini 2.5 Flash through API calls.
Data sent includes:

- Retrieved property context
- User-selected experience level
- Response preference
- Selected answer (during evaluation)

The AI response is returned in structured format and displayed to the user.

This mechanism ensures efficient retrieval, context accuracy, and smooth integration between storage, retrieval, and AI generation layers.

3.3 Data Retention Policies

The Data Retention Policy defines how long data is stored within the Real Estate Tutor Bot system and how it is managed.

1. Uploaded JSON Data

- Property JSON files uploaded by users are processed in-memory within the Streamlit application.
- The raw JSON files are not permanently stored unless explicitly configured in future deployments.
- Data exists only during the active session of the application.

2. Embedding Data in ChromaDB

- Property embeddings and associated metadata are stored in ChromaDB for retrieval during question generation.
- Since the application is currently not deployed in a production environment, embeddings persist only within the local storage environment.
- Data can be cleared manually if required.

3. User Interaction Data

- The system does not store user profiles, login credentials, or session history.
- No long-term storage of user responses or performance analytics is implemented in the current version.

4. Future Retention Considerations

If deployed in a production environment, the system may implement:

- Configurable retention periods
- Secure database storage
- Automated data deletion policies
- Compliance with data protection regulations

This policy ensures minimal data persistence while maintaining functionality for context-aware AI generation.

3.4 Data Migration

Data Migration defines how data would be transferred if the system is moved to another environment or upgraded in the future.

Currently, the application runs locally, and property embeddings are stored in ChromaDB within the local environment. If migration is required (e.g., to cloud deployment), the embedding data and metadata can be exported from ChromaDB and re-imported into a new database instance.

Since raw JSON files are not permanently stored and no user accounts exist, migration complexity is minimal. Future production deployment may require structured database backups, version control for embeddings, and secure data transfer mechanisms.

---

4. INTERFACES

4.1 User Interface

The system provides a web-based interface developed using Streamlit. It enables users to upload property listings in JSON format, select experience level and response type, generate multiple-choice questions, and view AI-generated feedback. The interface is designed to be simple, interactive, and user-friendly, ensuring smooth navigation throughout the learning process.

4.2 External Interfaces

The application interacts with external AI services through API calls.

- The Gemini 2.5 Flash API is used for question generation, answer evaluation, and feedback generation.
- The 001 Embedding Model API is used to convert property text data into vector embeddings.

These APIs are accessed programmatically within the Python application.

4.3 Database Interface

The system communicates with ChromaDB using internal library calls. This interface supports storing property embeddings with metadata and performing similarity-based retrieval during question generation. Users do not directly interact with the database layer.

4.4 Internal Interface

Internal interfaces define communication between different modules of the application, including the Streamlit UI module, data processing module, embedding generation module, vector database layer, and AI generation module. These components interact through structured Python function calls to ensure smooth data flow and modularity.

5. STATE AND SESSION MANAGEMENT

5.1 Application State Management

The Real Estate Tutor Bot manages application state within the Streamlit framework. Since Streamlit supports session-based interaction, temporary state information such as uploaded property data, generated embeddings, selected experience level, generated MCQs, and user-selected answers are maintained during the active session.

The system does not persist state beyond the runtime of the application unless explicitly stored in the vector database (embeddings only). All intermediate processing data remains in memory.

5.2 Session Handling

The current version of the system does not implement user authentication or multi-user session tracking. Each user interaction operates independently within the active Streamlit session.

There is:

- No login or account management
- No long-term tracking of user progress
- No persistent storage of user responses

If deployed in a production environment, session management could be extended to include:

- User authentication
- Performance tracking
- Personalized learning history
- Secure session handling mechanisms

6. CACHING

6.1 Overview

Caching in the Real Estate Tutor Bot is used to improve performance and reduce repeated computation, especially during embedding generation and AI model interaction. Since the application is built using Streamlit, caching mechanisms are handled at the application level.

6.2 Embedding Caching

When property data is uploaded and converted into embeddings, the generated vectors are stored in ChromaDB. This acts as a persistent cache layer, preventing the need to regenerate embeddings for the same property data repeatedly.

This improves:

- Response time during question generation
- Computational efficiency
- API cost optimization

6.3 Streamlit's Caching Mechanism

Streamlit's built-in caching utilities (such as st.cache_data or st.cache_resource) can be used to cache:

- Processed property data
- Database connections
- Embedding model initialization

This ensures that expensive operations are not executed multiple times during a session.

6.4 AI Response Caching (Optional Future Enhancement)

Currently, AI responses are generated dynamically. However, in future versions, caching previously generated MCQs or feedback responses could reduce repeated API calls and improve scalability.

6.5 Design Considerations

The caching strategy is designed to:

- Improve system performance
- Reduce redundant API calls
- Maintain consistency in retrieved context
- Support scalability for future deployment

---

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

The Real Estate Tutor Bot incorporates basic security measures appropriate for its current academic and local deployment environment.

- **Input Validation:** Uploaded JSON files are validated to ensure correct structure and prevent malformed data from affecting system stability.
- **Controlled API Access:** Interaction with external AI services (Gemini and Embedding API) is performed using secured API keys stored within the application environment.
- **Limited Data Storage:** The system does not store user credentials, personal information, or session history, reducing data exposure risks.
- **Restricted Database Access:** ChromaDB is accessed only through internal application logic; users have no direct access to the database layer.

For future production deployment, additional measures such as authentication, encrypted communication (HTTPS), role-based access control, and secure key management can be implemented.

7.2 Performance Aspects

The system is designed to provide efficient and responsive AI-driven interactions.

- **Efficient Retrieval Mechanism:** Use of vector embeddings and similarity search in ChromaDB ensures fast context retrieval.
- **Reduced Redundant Computation:** Embeddings are generated once and reused for subsequent queries.
- **Optimized Processing Flow:** The application runs within Streamlit, minimizing architectural overhead by avoiding unnecessary API layers.
- **Scalability Consideration:** The modular architecture allows future scaling of the vector database and AI services if deployed in a production environment.

Overall, the system balances computational efficiency with context-aware AI generation to maintain acceptable response times during MCQ generation and evaluation.

8. References

The following references were used in the design and implementation of the Real Estate Tutor Bot system:

1. Google AI. *Gemini API Documentation*. Available at: https://ai.google.dev
2. Google AI. *Embedding Model Documentation (001 Embedding)*. Available at: https://ai.google.dev/gemini-api/docs/embeddings
3. ChromaDB Documentation. *Chroma – Open Source Vector Database*. Available at: https://docs.trychroma.com
4. Streamlit Documentation. *Streamlit Official Documentation*. Available at: https://docs.streamlit.io
5. PlantUML Documentation. *PlantUML – Unified Modeling Language Tool*. Available at: https://plantuml.com

---